

Sistemas de renta de lockers para el área de ciencias químicas.

Manual del desarrollador

10/05/2026

por Manuel Alejandro Piña Becerril

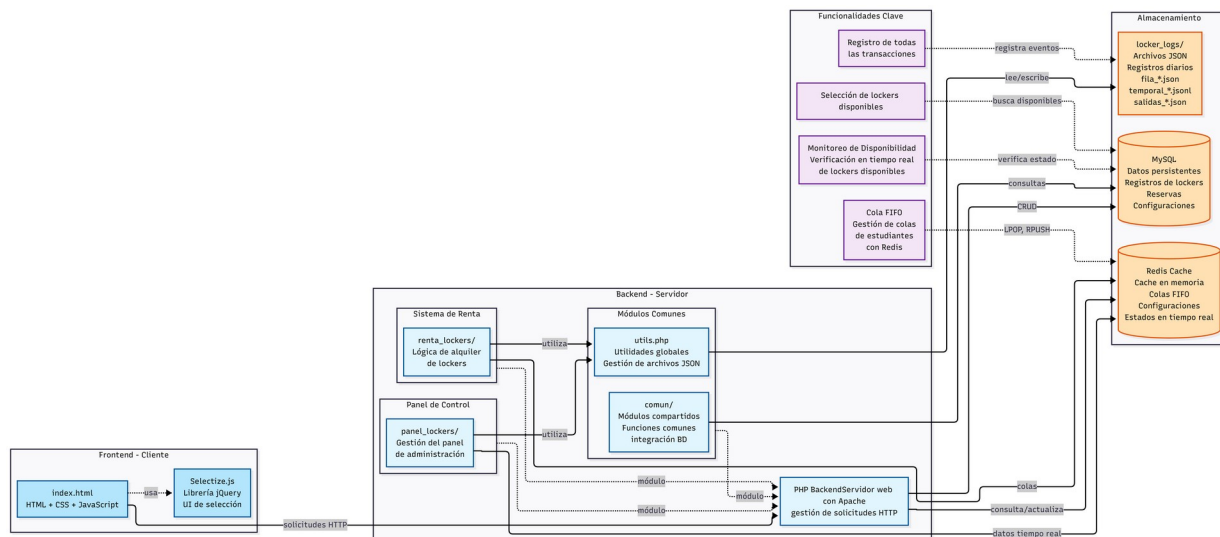
El propósito del sistema:

El presente proyecto surge ante la necesidad de optimizar el proceso de asignación de lockers para los estudiantes del área química. Anteriormente, la gestión de la cola de solicitudes se realizaba mediante consultas directas a SQL, lo que generaba una baja velocidad de respuesta y falta de estabilidad ante picos de alta demanda.

El presente sistema busca resolver este cuello de botella mediante la implementación de *Redis*. Al ser una base de datos en memoria, permite una gestión de datos de alta velocidad, garantizando un proceso eficiente, seguro y escalable, incluso bajo grandes volúmenes de solicitudes simultáneas."

Arquitectura del sistema:

En el siguiente diagrama se pueden apreciar los diferentes módulos que componen el sistema y cómo interactúan entre sí (todos los diagramas se pueden encontrar en la carpeta "Diagramas").



El sistema de renta de lockers fue desarrollado bajo una arquitectura monolítica modular, donde el frontend y el backend se encuentran integrados dentro de una misma aplicación web, pero organizados en módulos independientes para facilitar el mantenimiento y la escalabilidad.

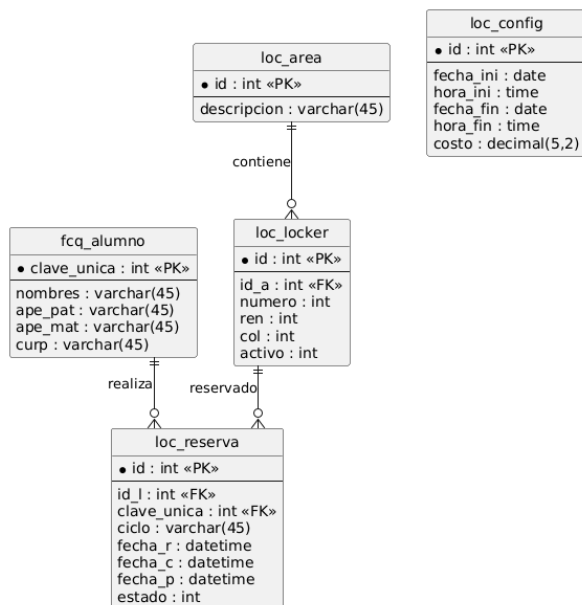
La interfaz del cliente está construida con HTML, CSS y JavaScript. En el servidor, el backend en PHP se encarga de procesar las solicitudes HTTP, administrar la lógica y coordinar los diferentes módulos del sistema, como el panel de administración, el sistema de renta y los módulos compartidos.

La arquitectura integra Redis como sistema de caché y manejo de colas FIFO en tiempo real, además de MySQL para el almacenamiento persistente de información relacionada con lockers, reservas y configuraciones.

El flujo de datos inicia cuando el usuario realiza una acción desde la interfaz web; dicha solicitud es enviada al servidor PHP, donde se procesa la lógica correspondiente y se consulta o actualiza información en Redis, MySQL o archivos JSON. Finalmente, el servidor genera una respuesta que es devuelta al cliente para actualizar la interfaz y mostrar la información solicitada en tiempo real.

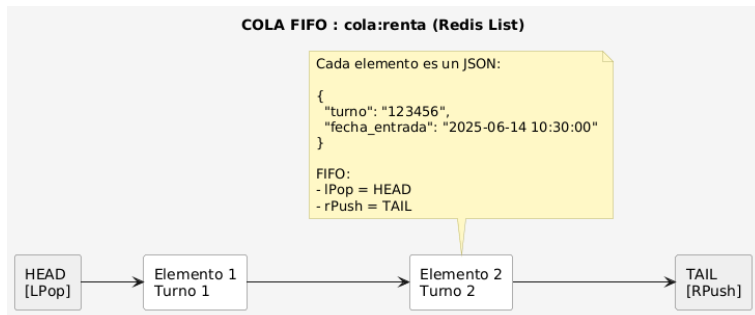
Base de datos:

El siguiente diagrama entidad-relación muestra las tablas y relaciones que son usadas en los archivos. Esta base de datos fue creada con la información proporcionada sobre la base de datos real, por lo que no debería haber problema al integrar el sistema completo. De haber algún problema, basta con cambiar los nombres de las tablas usadas en el sistema.



Fila de Redis

La fila de Redis usada en este proyecto para la renta de lockers está estructurada de la siguiente forma:



Estructura del Proyecto y Directorios:

El sistema panel_lockers está distribuido de la siguiente manera:

panel_lockers/

```
|— index.php                # Redireccionamiento a la página principal
|— interfaz_atender_alumno.php # Página principal del panel
|— redis/
|   |— cola/
|   |   |— atender_alumno.php # Mueve al siguiente alumno en la cola manualmente
|   |— comun/
|   |   |— conexion_redis.php # Inicia la conexión con Redis en el puerto 6379
|   |   |— utils.php          # Funciones relacionadas con los archivos json
|   |   |— verificar_sistema.php # Verifica si el sistema esta en funcionamiento
|   |   |— procesos.php       # Funciones relacionadas con la cola automática
|   |— consultas/
|   |   |— obtener_detalle_filas.php # Obtiene la información de la cola activa
|   |   |— obtener_todos_registros.php # Obtiene los registros del ciclo
|   |— sistema/
|   |   |— abrir_sistema.php # Abre sistema (acepta nuevas entradas)
|   |   |— auto_abrir_cerrar.php # Encargado de abrir y cerrar automáticamente
|   |   |— cola_automatica.php # Encargado de mover la cola automáticamente
|   |   |— cerrar_sistema.php # Cierra sistema (vacía cola)
|— ajax/
|   |— obtener_reservas.php # Lista todas las reservas
|   |— actualizar_estado_reserva.php # Modifica una reserva
|   |— reporte_todos_registros_xlsx.php # Genera un xlsx con los registros
|   |— guardar_config.php # Sube la configuración del sistema
|   |— obtener_config.php # Obtiene la configuración del sistema
|— js/
|   |— fila_virtual.js # Gestión de cola - eventos y AJAX
|   |— renta_locker.js # Tabla de reservas y acciones
|   |— configuración.js # Gestión de la configuración
```

El sistema renta_lockers está distribuído de la siguiente manera:

renta_lockers/

- |— index.php # Redireccionamiento a la página principal
- |— login.php # Inicio de sesión usando Clave única y CURP
- |— dashboard.php # Módulo para interactuar con la fila
- |— cerrar_sesion.php # Acabar con la sesión del navegador
- |— seleccionar_locker.php # Módulo para la selección de locker
- |— ajax/
 - | |— actualizar_locker_json.php # Modificar información de los lockers en el json
 - | |— cancelar_reserva.php # Modifica una reserva previamente hecha
 - | |— cancelar_seleccion_locker.php # Saca al usuario de la selección de locker
 - | |— expulsar_seleccion.php # Cuando el tiempo acaba saca al usuario
 - | |— reservar_locker.php # Completa la reserva de lockers
- |— js/
 - | |— seleccionar_locker.js # Funciones para la selección de locker
- |— redis/
 - | |— cola/
 - | | |— consultar_estado.php # Regresa la posición en fila o tiempo restante
 - | | |— salir_cola.php # Saca al alumno de la fila si así lo quiere
 - | | |— unirse_cola.php # Mete al alumno al alumno a la fila y le da turno
 - | |— comun/
 - | | |— conexion_redis.php # Inicia la conexión con Redis en el puerto 6379
 - | | |— verificar_sistema.php # Verifica si el sistema esta en funcionamiento

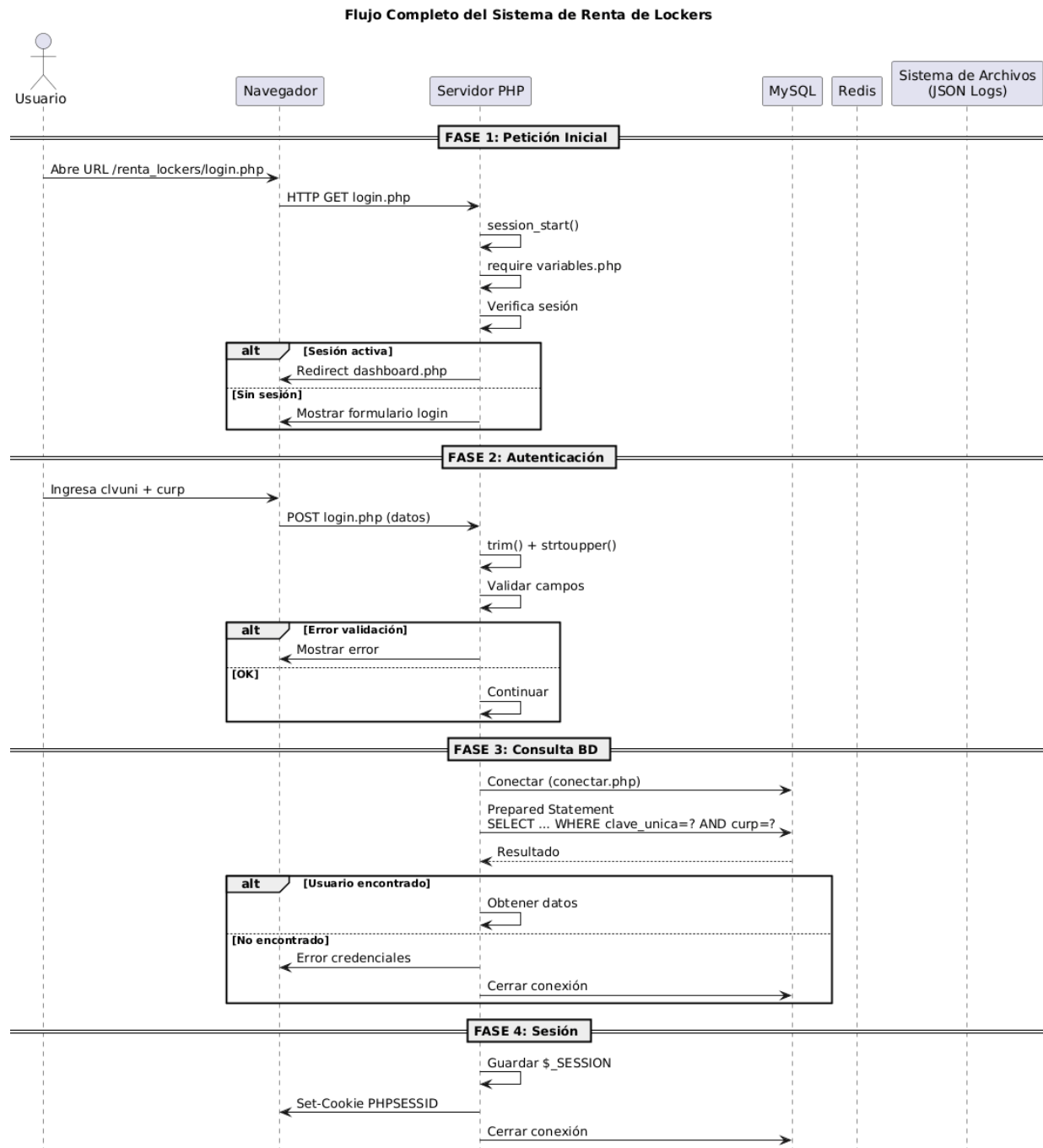
La carpeta locker_logs está distribuída de la siguiente forma:

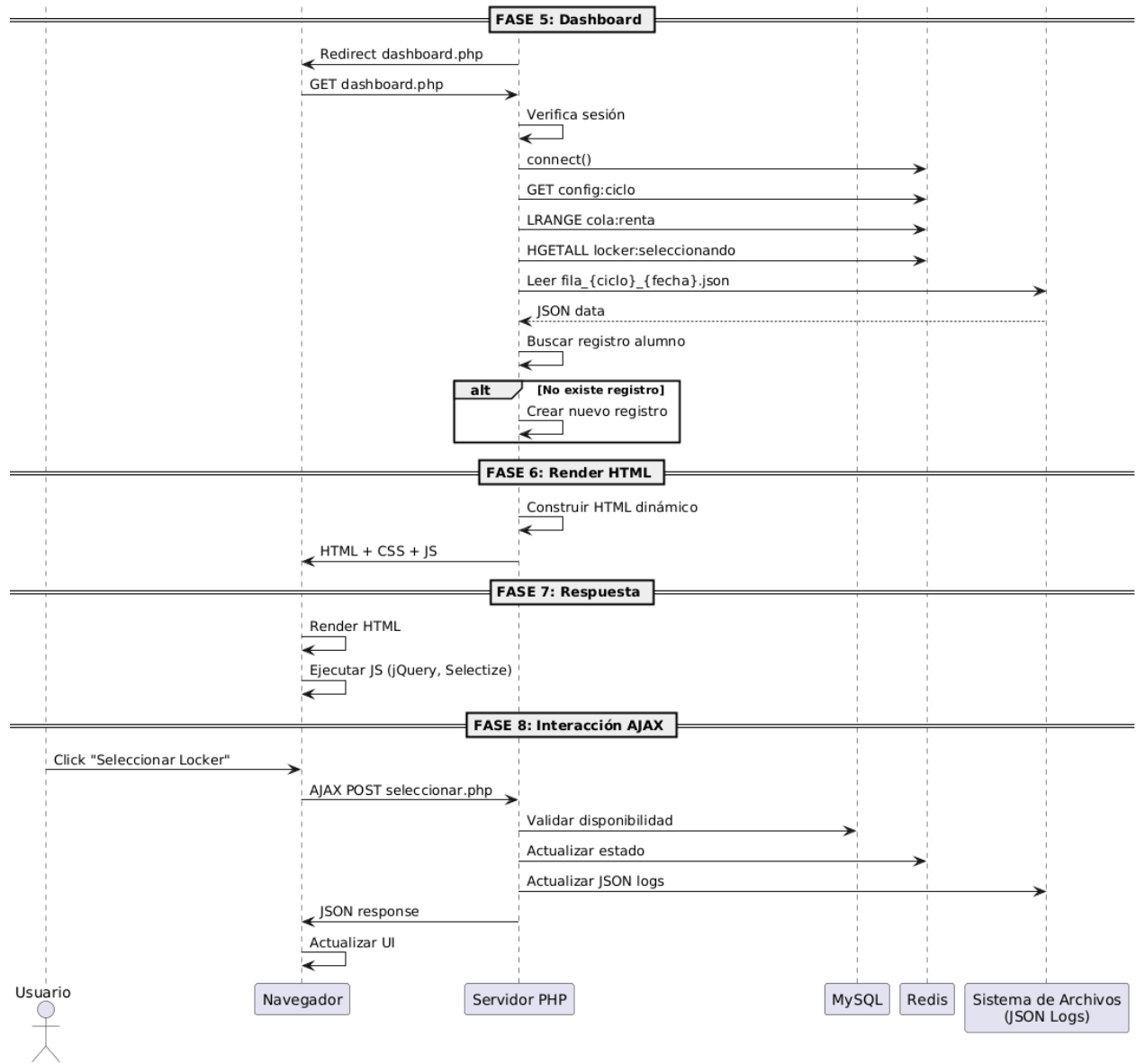
locker_logs/

- |— fila_ciclo_fecha.json # Registro de los movimientos de los alumnos
- |— registro/
 - | |— turnos_perdidos_fecha.json # Registro de los turnos desaparecidos
- |— temp/
 - | |— salidas_ciclo_fecha.jsonl # Registro de las personas que salieron
 - | |— temporal_ciclo_fecha.jsonl # Registros temporales del día

Flujo del sistema de lockers:

En este diagrama se muestra el flujo normal de un usuario en el sistema





Stack Tecnológico y Dependencias:

Backend

- PHP 7.4+

- Razón: Lenguaje interpretado que facilita desarrollo rápido. Soporte nativo para sesiones y manejo de cookies.

Frontend

- HTML5+ CSS3+ JavaScript (Vanilla + jQuery 3.4.1)

- Razón: jQuery proporciona compatibilidad cruzada de navegadores y AJAX simplificado

Base de Datos

- MySQL

- Razón: Motor SQL robusto, soporte de transacciones, integridad referencial

- Contiene tablas críticas:

- `plantilla.fcq\_alumno` - Datos de estudiantes (autenticación)

- `plantilla.loc\_reserva` - Registro de reservas

- `plantilla.loc\_locker` - Catálogo de lockers

- `plantilla.loc\_config` - Configuración de horarios

Cache y Colas

- Redis 6.0+

- Razón: Alto rendimiento para colas FIFO, almacenamiento de configuración, control de estado del sistema

Servidor Web

- Apache

- Módulo PHP habilitado

Justificación de Decisiones Tecnológicas:

¿Por qué Redis en lugar de base de datos SQL?

- Redis: Operaciones atómicas en microsegundos para colas
- SQL: Transacciones lentas para operaciones frecuentes
- Solución: Redis para estado real-time, MySQL para persistencia

¿Por qué archivos JSON en lugar de base de datos para logs?

- JSON: Legibilidad, auditoría manual, recuperación sin consultas complejas
- Alternativa: Podría usarse tabla en MySQL pero impactaría el rendimiento
- Ventaja: Permite análisis histórico sin sobrecargar BD principal

¿Por qué PHP y no un framework moderno?

- Requisito institucional: Compatibilidad con infraestructura FCQ existente
- Ventaja de PHP: Sesiones nativas, integración simple con MySQL

Información relevante:

Conexión a Redis en puerto 6379

Estados manejados redis:

- config:estado_sistema → "abierto" | "cerrado"
- config:ciclo → Ciclo escolar actual
- config:fecha_apertura → Timestamp de apertura
- contador:turno → Número incremental de turnos
- locker:seleccionando → Hash con usuarios siendo atendidos
- locker → Cola FIFO de usuarios esperando

Estados manejados en archivos JSON sobre los alumnos:

- **0:** En cola
- **1:** Salió de la cola
- **2:** Seleccionando locker
- **3:** Terminó correctamente
- **4:** Fue expulsado por tiempo
- **5:** Canceló su locker o la selección

Estados manejados en base de datos para las reservas:

- **0:** Vacío
- **1:** Reservado
- **2:** Sin Pago
- **3:** Pagado

Manejo de Errores y Problemas Conocidos

Instalación:

Uno de los principales problemas se presenta durante la instalación y tiene que ver con la asignación de permisos. Esto se soluciona agregando las carpetas y archivos a los grupos correctos de apache.

- `sudo chown -R www-data:www-data /var/www/html/(carpetas)`
- `sudo chown -R 775 /var/www/html/(carpetas)`

Problemas con la cola:

El sistema se abre en cuanto el primer alumno detecta que está en el horario. Esto corre un archivo php del lado del servidor que se sigue ejecutando hasta terminar el horario establecido. Si el sistema funciona correctamente, al ejecutar el siguiente comando se debe mostrar que el archivo `cola_automatica.php` está corriendo:

- `ps aux | grep php`

Este proceso se puede terminar en caso de que esté dando problemas o se quiera forzar el cierre del sistema. Se volverá a generar en el momento en que un alumno nuevo entre al sistema.

Un comando útil para saber es el que te regresa la variable que indica si el sistema está abierto o cerrado (la variable solo tiene esos dos valores):

- `redis-cli GET config:estado_sistema`

El siguiente comando en la consola ayuda a visualizar todos los alumnos que están actualmente en la fila formados. Todos deben estar en orden:

- `redis-cli LRANGE lockers 0 -1`

La variable incremental con el valor de los turnos se consulta con el siguiente comando:

- `redis-cli GET contador:turno`

El siguiente comando en la consola muestra todos los alumnos que actualmente están seleccionando un locker.

Si el sistema está abierto y hay lockers disponibles siempre debería tener valor. De lo contrario significa que la cola automática nunca se inició.

- `redis-cli HGETALL locker:seleccionando`

Problemas con los archivos json:

Estos archivos son generados en cuanto el primer alumno abre el sistema. Si no están en la carpeta los correspondiente al día presente, significa que no fueron creados. El problema está asociado a los permisos de creación y modificación.

Turnos perdidos:

La asignación de turnos funciona con una variable incremental en redis. De esta manera, los turnos no se repiten. Sin embargo, para mantener el orden en los turnos, antes de entrar se verifica que el turno anterior está formado. Este proceso es bastante rápido, por lo que no debería causar ningún problema.

Aun así, hay un caso concreto donde sí es posible que se surja un problema, y es cuando la última persona en la cola se sale totalmente de la cola, interfiriendo en la asignación del turno consecutivo.

Esto actualmente está solucionado de la siguiente forma: Cuando una persona no encuentra el turno anterior espera 15 segundos. Si la persona no se ha incorporado, se fuerza la entrada del usuario a la cola, continuando el flujo normal.

Todas estas ocurrencias son almacenadas en la carpeta /registro. En un json se coloca la información del turno que está “perdido” de la cola.

Abrir y Cerrar el sistema Manualmente:

Desde el panel de administrador hay dos botones: Abrir sistema y Cerrar sistema. Estos botones actualmente se encuentran ocultos (comentados en el código). Sin embargo, pueden volverse a mostrar para usarlos en casos de emergencia y así forzar la apertura o cierre.